

Code for ‘Targeted Maximum Likelihood Estimation of Vaccine Effectiveness and Immune Correlates in Test-Negative Design Studies with Missing Data’

Application 1: Inference on COVID-19 Vaccine Effectiveness

Leah I. B. Andrews

April 14, 2026

This R markdown provides code to perform the analyses from ‘Application 1: Inference on COVID-19 Vaccine Effectiveness’ of the manuscript “Targeted maximum likelihood estimation of vaccine effectiveness and immune correlates in test-negative design studies with missing data” by L.I.B. Andrews, L. van der Laan, and P. Gilbert. This manuscript introduces a targeted maximum likelihood estimation approach to evaluate COVID-19 vaccine effectiveness in a test-negative design (TND) study cohort. We demonstrate our approach on a TND study cohort sampled from the Moderna Coronavirus Efficacy (COVE) phase 3 randomized placebo-controlled trial (RCT) (El Sahly et al., 2021).

The COVE RCT data cannot be shared so instead, we generate a toy RCT dataset to illustrate how our analyses in the manuscript. We apply participant-based sampling with censoring for COVID-19 to the toy RCT dataset to obtain our TND study cohort (De Serres et al., 2013 and Andrews et al., 2025). We then calculate our targeted maximum likelihood estimator (TMLE) and an ordinary logistic regression estimator of TND vaccine effectiveness against symptomatic COVID-19 in the healthcare-seeking population using the TND study cohort. We also estimate RCT vaccine efficacy using a stratified Cox proportional hazards regression model on the entire RCT cohort for comparison.

Generating Data

Since we cannot share the COVE data, we generate a toy phase 3 COVID-19 RCT cohort and identify RCT participants who would have been eligible for a TND study from this RCT cohort. This data-generating code has been adapted from “Evaluating the Test-Negative Design for COVID-19 Vaccine Effectiveness Using Randomized Trial Data: A Secondary Cross-Protocol Analysis of 5 Randomized Clinical Trials” by Andrews et al. in JAMA Network Open in 2025 (<https://github.com/leahandrews/eval-tnd-w-rcts>).

RCT Cohort

Our toy RCT dataset consists of 30,000 participants who were randomly assigned to receive one dose of the COVID-19 vaccine or placebo at a 1:1 ratio within three strata. The blinded phase of the RCT takes place from August 1, 2020 to March 31, 2021. For simplicity, all participants were vaccinated in the first two weeks of August and no participants were unblinded early or lost to follow-up. At RCT enrollment, age, sex at birth, race/ethnicity, geographic region, and presence of any comorbidities were measured for all participants. There are no missing data on any covariates.

```

## Demographic and Clinical Characteristics for RCT Cohort
# One Row per Participant
set.seed(7)
nsubj <- 30000

demo.df00 <- data.frame(USUBJID = paste0("ID", 1:nsubj),
  AGE = round(runif(nsubj, min = 18, max = 80)),
  SEX = sample(c("F", "M"), nsubj, replace = TRUE),

  # Dichotomized Race/Ethnicity (Hispanic/Latino or Non-White vs.
  # Non-Hispanic/Non-Latino White)
  POC = sample(c("POC", "White"), nsubj,
    replace = TRUE, prob = c(1/3, 2/3)),

  # US Census Regions
  REGION = sample(c("Midwest", "NE", "South", "West"), nsubj,
    replace = TRUE, prob = c(2/10, 1/10, 5/10, 2/10)),

  # Presence of Comorbidities
  COMORB = sample(c("Comorbidities", "None"), nsubj,
    replace = TRUE, prob = c(1/4, 3/4)),

  # Intervention Date (Placebo or Vaccine)
  DOSEDT = sample(seq(as.Date('2020/08/01'),
    as.Date('2020/08/14'), by="day"),
    nsubj, replace = TRUE),

  # End of Blinded Phase Date
  ENDDT = as.Date('2021/03/31')) %>% mutate(

  # Randomization Strata
  STRATA = case_when(
    AGE < 65 & COMORB == "Comorbidities" ~ "< 65 and Comorb",
    AGE < 65 & COMORB == "None" ~ "< 65 and No Comorb",
    AGE >= 65 ~ ">= 65")
)

demo.df0 <- demo.df00 %>% group_by(STRATA) %>%
  mutate(#1:1 Vaccine vs. Placebo by Randomization Strata
    VACCINE = sample(rep(0:1, length.out = n()))) %>% ungroup()

```

During the RCT, participants may experience an illness episode or symptomatic period that prompts them to obtain SARS-CoV-2 testing. In this dataset, participants can experience 0-5 illness episodes during the study and the date of any symptom onset (ONSETDT) and first date of meeting the COVID-19 symptom definition (SYMDET) are documented. By definition, the date of meeting the symptom definition must occur on or after date of symptom onset. Illness episodes may be truly caused by SARS-CoV-2 or by a non-SARS-CoV-2 illness (e.g., other respiratory pathogens, allergies). By construction, the COVID-19 vaccine protects against COVID-19 (vaccine efficacy = 80%) and does not affect non-SARS-CoV-2 illness.

Participants obtain 1-10 SARS-CoV-2 diagnostic tests before and/or during an illness episode. While an illness episode may be truly caused by SARS-CoV-2 (`eps_sars_inf=1`), our SARS-CoV-2 test results (POSTEST) are subject to misclassification (100% specificity, 85% sensitivity). Thus, the unobserved true SARS-CoV-2 infection status of an illness episode, `eps_sars_inf`, does not always match the observed SARS-CoV-2 test results from an illness episode, POSTEST.

```

# Adding Illness Episode Information
demo.df <- demo.df0 %>% mutate(
  # True SARS-CoV-2 Infection Status at Any Point during Blinded Phase
  # (Unknown in Observed Clinical Data)
  # Vaccine Protects Against SARS-CoV-2 Infection
  any_sars_inf = rbinom(nrow(.), 1, exp( log(0.04) + log(0.2)*VACCINE) ),

  # Number of COVID-19 Illness Episodes
  n_sars_eps = any_sars_inf*sample(1:2, nsubj, replace= TRUE, prob = c(0.95, 0.05)),

  # Number of Non-COVID-19 Illness Episodes
  n_nonsars_eps = sample(0:3, nsubj, replace = TRUE, prob = c(0.9, 0.07, 0.02, 0.01)))

## Illness Episode Characteristics
# Creating One Row Per Illness Episode

# COVID-19 Illness Episodes
sars.eps.df0 <- uncount(demo.df , n_sars_eps) %>%
  # Illness Episode SARS-CoV-2 Infection Status
  mutate(eps_sars_inf = 1) %>% select(-n_nonsars_eps)

# Non-COVID-19 Illness Episodes
non.sars.eps.df0 <- uncount(demo.df , n_nonsars_eps) %>%
  # Illness Episode SARS-CoV-2 Infection Status
  mutate(eps_sars_inf = 0) %>% select(-n_sars_eps)

# Adding Testing and Symptom Information for Both Types of Illness Episodes
eps.df00 <- rbind(sars.eps.df0, non.sars.eps.df0) %>% arrange(USUBJID)
eps.df0 <- eps.df00 %>% mutate(
  # Number of SARS-CoV-2 Tests within an Illness Episode
  ntests = sample(1:10, nrow(.), replace = TRUE),

  # Date of Symptom Onset Beginning of Illness Episode)
  ONSETDT = sample(seq(as.Date('2020/09/01'), as.Date('2021/03/31'),
    by="day"), nrow(.), replace = TRUE),

  # Date of Meeting the COVID-19 Symptom Definition
  # For Simplicity, All Our Illness Episodes Meet the
  # Symptom Definition But This May Not be True in Other Datasets
  SYMDT = ONSETDT + sample(0:5, n(), replace = TRUE))%>%
  arrange(USUBJID, ONSETDT)

eps.df <- eps.df0 %>% group_by(USUBJID) %>% arrange(ONSETDT) %>%
  # Label Illness Episodes in Chronological Order
  mutate(EPISODE = 1:n()) %>% arrange(USUBJID)

## Testing Characteristics
# Creating One Row per SARS-CoV-2 Test Result
test.df0 <- uncount(eps.df, ntests)
test.df <- test.df0 %>% group_by(USUBJID)%>% mutate(
  # Test Date

```

```
TESTDT = ONSETDT + sample(-3:15, n(), replace = TRUE),
# Test Result Roughly Based on SARS-CoV-2 PCR Sensitivity and Specificity
# 1 = positive, 0 = negative
POSTEST = eps_sars_inf*rbinom(n(), 1, 0.85)) %>%
# Only Include Tests while Blinded
filter(TESTDT <= as.Date('2021/03/31'))
```

We use the demographic and testing data to identify RCT COVID-19 cases (RCTCASE=1), or participants who meet the COVID-19 symptom definition and obtain a SARS-CoV-2 positive test at least 14 days after vaccination. For this RCT, these criteria can be met in either order as long as both dates are within 15 days apart. The remaining RCT participants were right-censored at the end of the blinded phase.

```
## Creating RCT Analysis Dataset

# Identify First SARS-CoV-2 Positive Test from RCT COVID Cases
# (In this Example Dataset, All Illness Episodes Met the Symptom Definition)
rct.case.df <- test.df %>%
  # Only take those who test SARS-CoV-2 Positive
  filter(POSTEST == 1) %>% group_by(USUBJID)%>%
  slice_min(TESTDT, with_ties = FALSE)

# Adding COVID-19 Cases' Testing Information to Demographic Information
rct.df0<- left_join(demo.df, rct.case.df[c("USUBJID", "SYMDT", "TESTDT", "POSTEST")])
rct.df <- rct.df0 %>% rowwise() %>% mutate(
  # Endpoint Date: Unblinding Date or
  # Earliest Date of Meeting Symptom Definition and Positive Test
  RCTADT = if_else(is.na(POSTEST), ENDDT, min(SYMDT, TESTDT))) %>%
  ungroup() %>% mutate(
  # RCT COVID-19 Case Status
  RCTCASE= ifelse(is.na(POSTEST), 0, 1),
  # Time to COVID-19 or Censoring
  RCTTT = as.numeric(RCTADT - (DOSEDT + 14) + 1))

# RCT Cases
rct_case_ids<- subset(rct.df, RCTCASE == 1)$USUBJID

with(rct.df, table(VACCINE, RCTCASE))
```

```
##          RCTCASE
## VACCINE      0      1
##          0 14412   589
##          1 14866   133
```

From the RCT cohort of 30,000 participants, 722 participants were COVID-19 cases.

TND Study Cohort

To create our TND study cohort from the RCT data, we only retain participants with eligible SARS-CoV-2 tests, which must occur at least 14 days after vaccination, within ten days after symptom onset, after meeting the symptom definition, and while blinded. In the TND analysis, we define illness episodes as symptomatic periods that last from symptom onset to at least 15 days after symptom onset and are separated by eligible SARS-CoV-2 tests that occur at least 30 days apart. We propose distinguishing illness episodes by eligible

SARS-CoV-2 test dates rather than dates of symptom onset and resolution because symptom dates would be more difficult to recall and collect in a TND study. Since the illness episode variable EPISODE is defined differently in the RCT, we apply a stricter criteria to the existing RCT illness episode EPISODE variable. We define a positive episode as an illness episode that triggers at least one eligible SARS-CoV-2 positive test and a negative episode as an illness episode that does not trigger any SARS-CoV-2 positive tests and only triggers eligible SARS-CoV-2 negative tests.

```
# Only Retain TND Eligible Tests
tnd.test.df <- test.df %>% filter(# Test At Least 14 Days After Vaccination
  TESTDT >= (DOSEDT+14) &

  # Test within 10 Days After Symptom Onset
  TESTDT >= ONSETDT &
  TESTDT <= (ONSETDT +10) &

  # Test After Meeting the Symptom Definition
  TESTDT >= SYMDT &

  # Test While Blinded
  TESTDT <= ENDDT) %>%

mutate(# Quantitative Calendar Time from Sept 1, 2020 Reference Date
  # (for TMLE Approach)
  CALTIME = as.numeric(TESTDT - as.Date("2020-09-01")),

  # Two Week Calendar Time
  # (Many TND Analyses Adjust for Categorical Calendar Time)
  TWOWEEK0 = cut(TESTDT, "2 weeks"))

# Function that Identifies Tests That are at Least 30 days from Each Other
# Must be Applied to Data Grouped by Participant ID and Sorted by Testing Date
# Inputs:
#   col_vals: Values from a Date or Numeric Column
# Outputs: Boolean Vector, TRUE meaning rows are at least 30 days apart
#   First Value will Always be True
#
over30_func <- function(col_vals){
  # MUST BE USED AFTER GROUPED BY SUBJID AND SORTED BY DATE

  # starts with TRUE because have to keep first test
  empty_vec <- c(TRUE)
  nobs <- length(col_vals)
  if(nobs > 1){
    for(i in 1:(nobs - 1)){
      # Are Two Tests at Least 30 Days Apart?
      val <- col_vals[i + 1] - col_vals[i] >= 30
      empty_vec <- c(empty_vec, val)
    }
  }
  empty_vec
}
```

Take One Test per RCT-Defined Illness Episode
(Earliest Negative Test if All Negative Tests, or Earliest Positive Test)

```
spec.df0 <- tnd.test.df %>% group_by(USUBJID, EPISODE)%>%
  arrange(USUBJID, TESTDT) %>% slice(which.max(POSTEST))

# Only Retain Illness Episodes That Are at Least 30 Days Apart
spec.df01 <- spec.df0 %>% group_by(USUBJID) %>%
  arrange(TESTDT)%>% mutate(SEPILL = over30_func(TESTDT))
spec.df02 <- spec.df01 %>% filter(SEPILL==TRUE)

# Only Retain Earliest Positive Episode
spec.df <- spec.df02[!(duplicated(spec.df02[c("USUBJID", "POSTEST")])) &
  spec.df02$POSTEST==1), ] %>%
  mutate(TWOWEEK = droplevels(TWOWEEK0))
```

To obtain our TND study cohort, we apply participant-based sampling with censoring for COVID-19. Under this TND sampling method, cases are participants with at least one positive episode and contribute their earliest eligible SARS-CoV-2 positive test from their first positive episode. Noncases are participants with no positive episodes and only negative episodes and contribute their earliest eligible SARS-CoV-2 negative test from their first negative episode.

```
# Constructing TND Study Cohort
part.wcc.df0 <- spec.df %>% group_by(USUBJID) %>% arrange(USUBJID, TESTDT) %>%
  slice(which.max(POSTEST))

part.wcc.df <- part.wcc.df0 %>%
  mutate(TWOWEEK = droplevels(TWOWEEK0))

with(part.wcc.df, table(VACCINE, POSTEST))
```

```
##          POSTEST
## VACCINE      0      1
##          0 1390  471
##          1 1289  107
```

Our TND study cohort has 3257 participants and 578 cases.

Statistical Analyses

In this next section, we estimate vaccine efficacy against COVID-19 in the RCT cohort and vaccine effectiveness against COVID-19 using two statistical methods in the TND study cohort.

RCT COVID-19 Vaccine Efficacy

```
# Cox PH Model
rct.mod <- coxph(Surv(RCTTT, RCTCASE) ~ VACCINE + strata(STRATA), data = rct.df)
rct.ve <- round(1 - summary(rct.mod)$conf.int["VACCINE", "exp(coef)"], 3)*100
rct.veCIlow <- round(1 - summary(rct.mod)$conf.int["VACCINE", "upper .95"], 3)*100
rct.veCIhigh <- round(1 - summary(rct.mod)$conf.int["VACCINE", "lower .95"], 3)*100
```

From the RCT cohort of 3×10^4 participants, we estimate vaccine efficacy, defined as 1 minus the hazard ratio (vaccine vs. placebo), using a Cox proportional hazards (PH) regression model stratified by strata used for randomization (< 65 years old with comorbidities, < 65 years old without comorbidities, and ≥ 65 years old). RCT vaccine efficacy against COVID-19 is 77.8% (95% CI: 73.2, 81.6).

TND COVID-19 Vaccine Effectiveness Using Targeted Maximum Likelihood Estimation Under Semiparametric Logistic Regression Model

To calculate our TMLE, we use the `causalglm` R package developed by Lars van der Laan (<https://tlverse.org/causalglm/>), which implements semiparametric and nonparametric generalized linear models using targeted maximum likelihood estimation to conduct causal inference on heterogeneous treatment effects. The package relies on `tlverse/tmle3` for targeted maximum likelihood estimation and `tlverse/sl3` and `tlverse/hal9001` for machine-learning. The package is in active development so please contact Lars van der Laan if there are any issues or concerns (lvdlaan@uw.edu). Make sure to load the most up-to-date version of these packages from Github.

```
# Install devtools CRAN package
if(!require(devtools)) {
  install.packages(devtools)
}

# Install Github Packages
devtools::install_github("tlverse/causalglm")
devtools::install_github("tlverse/hal9001@master")
devtools::install_github("tlverse/tmle3@general_submodels_devel")
devtools::install_github("tlverse/sl3@Larsvanderlaan-formula_fix")
```

While our data does not have confounding, to mimic TND studies that must account for confounding, we fit our TMLE of vaccination status adjusted for SARS-CoV-2 status, age, sex assigned at birth, race/ethnicity, region, comorbidities, and quantitative calendar time. We use a 10-fold cross-fitted ensemble super learner that considers a library of learners including the sample mean, logistic regression models, general additive models, multivariate adaptive regression splines, lasso regression, ridge regression, highly adaptive lasso, random forests, and gradient boosting for estimation. The library also includes screened logistic regression and general additive model learners, which only include covariates with nonzero coefficients based on lasso regression. We assess the candidate learners using a binomial log-likelihood loss and 10-fold cross-validation and combine the nuisance function predictions using non-negative least squares regression.

```
## Ensemble Super Learner

# Define Learners without Screens
mean_lrn <- Lrn_r_mean$new()
glm_lrn <- Lrn_r_glm$new(family = binomial())
glm_intx <- Lrn_r_glm$new(formula = ~ (. )^2, family = binomial())
gam_lrn <- Lrn_r_gam$new(family = binomial())
mars_lrn <- Lrn_r_earth$new()
lasso_lrn <- Lrn_r_glmnet$new(alpha = 1, family = "binomial")
ridge_lrn <- Lrn_r_glmnet$new(alpha = 0, family = "binomial")
ranger_lrn <- Lrn_r_ranger$new(probability = TRUE)
xgb3 <- Lrn_r_xgboost$new(verbose = 0, max_depth = 3,
  objective = "binary:logistic")
xgb5 <- Lrn_r_xgboost$new(verbose = 0, max_depth = 5,
  objective = "binary:logistic")
hal3_lrn <- Lrn_r_hal9001$new(family = binomial(),
```

```

                                smoothness_orders = 1,
                                max_degree = 1,
                                num_knots = 3)
hal3x_lrn <- Lrnr_hal9001$new( family = binomial(),
                                smoothness_orders = 1,
                                max_degree = 2,
                                num_knots = 3)
hal10_lrn <- Lrnr_hal9001$new(family = binomial(),
                                smoothness_orders = 1,
                                max_degree = 1,
                                num_knots = 10)
hal10x_lrn <- Lrnr_hal9001$new( family = binomial(),
                                smoothness_orders = 1,
                                max_degree = 2,
                                num_knots = 10)

# Define Learners with Lasso Screen
lasso_screen <- Lrnr_screener_coefs$new(learner =
                                Lrnr_glmnet$new(alpha = 1, family = "binomial"), threshold = 0)
glm_lasso <- Pipeline$new(lasso_screen, glm_lrn)
glm_intx_lasso <- Pipeline$new(lasso_screen, glm_intx)
gam_lasso <- Pipeline$new(lasso_screen, gam_lrn)

# Stack Learners
ve_stack <- Stack$new(
  mean_lrn,
  glm_lrn,
  glm_intx,
  gam_lrn,
  mars_lrn,
  lasso_lrn,
  ridge_lrn,
  ranger_lrn,
  xgb3,
  xgb5,
  hal3_lrn,
  hal3x_lrn,
  hal10_lrn,
  hal10x_lrn,
  glm_lasso,
  glm_intx_lasso,
  gam_lasso)

# SuperLearner
ve_sl <- Lrnr_sl$new(
  learners = ve_stack,
  metalearner = Lrnr_nnls$new(),
  loss_function = loss_loglik_binomial)

```

To run a semiparametric logistic regression of vaccination status, we use the `spglm` function in `causalglm`, which currently requires all variables in the input dataset to be numerics. We use `model.matrix` to convert all character and factor variables into numeric or indicator variables. We also standardize quantitative variables, like age and calendar time.


```

# Standardize Variables
part.wcc.df <- part.wcc.df %>% dplyr::mutate(
  AGES = (AGE - mean(part.wcc.df$AGE))/ sd(part.wcc.df$AGE),
  CALTIMES = (CALTIME - mean(part.wcc.df$CALTIME))/
    sd(part.wcc.df$CALTIME))

# Convert Data to Only Contain Numeric/Indicator Variables (spglm Requirement)
# Use model.matrix to Convert Character/Factor Variables for Each TND Sample
part.wcc.semi.model.df <- as.data.frame(model.matrix(~ VACCINE +
  POSTEST + AGES + SEX + POC + REGION +
  COMORB + CALTIMES, data = part.wcc.df))

# Run TMLE Approach with Super Learner for Estimation
t1 <- Sys.time()
set.seed(2025)
semi.mod <- spglm( ~1, part.wcc.semi.model.df,
  W = setdiff(names(part.wcc.semi.model.df),
    c("(Intercept)", "VACCINE", "POSTEST")),
  A = "POSTEST", Y = "VACCINE",
  estimand = "OR",
  sl3_Learner_Y = ve_sl,
  sl3_Learner_A = ve_sl)

t2<- Sys.time()
t2 - t1 # takes 40 minutes

saveRDS(semi.mod,
file = "~/Desktop/UW Stuff/IS TND/Code/TND TMLE/semiparametric-tmle-tnd/Data Applications/COVID VE TMLE

# Load Saved File to Save Time
semi.mod <- readRDS(file =
"~/Desktop/UW Stuff/IS TND/Code/TND TMLE/semiparametric-tmle-tnd/Data Applications/COVID VE TMLE Super I

# Model Output
print(semi.mod)

## A causalglm fit object obtained from spglm for the estimand OR with formula:
## log OR(W) = -1.4 * (Intercept)

# Example Output
summary(semi.mod)

## A causalglm fit object obtained from spglm for the estimand OR with formula:
## log OR(W) = -1.4 * (Intercept)
##
## Coefficient estimates and inference:
##      type      param  tmle_est      se      lower      upper  psi_exp
##   <char>    <char>    <num>    <num>    <num>    <num>    <num>
## 1:      OR (Intercept) -1.401296 0.1137727 -1.624287 -1.178306 0.2462775
##   lower_exp upper_exp  Z_score p_value
##        <num>    <num>    <num>    <num>
## 1: 0.1970522 0.3077998 12.31663      0

```

Using our library of learners, our TMLE approach takes 40 minutes so we ran it beforehand and loaded into this document to minimize the compilation time. `print` of an `spglm` object reports the object and equation. `summary` of an `spglm` object reports summary statistics for the sample log conditional odds ratio of vaccination by case status, adjusting for covariates. `tmle_est` is the log conditional odds ratio and `psi_exp` is the conditional odds ratio. `lower_exp` and `upper_exp` are the exponentiated lower and upper bounds of the log conditional odds ratio 95% confidence interval.

Assuming the eleven identifying assumptions specified in our manuscript hold, we can interpret the TND sample conditional odds ratio as the causal conditional risk ratio of COVID-19 comparing vaccinated to unvaccinated healthcare-seeking individuals with the same covariates. Then, calculating 1 minus the causal conditional risk ratio, we estimate that the TND vaccine effectiveness against symptomatic COVID-19 in healthcare-seeking individuals with the same covariates is 75.4% (95% CI: 69.2, 80.3) using the targeted maximum likelihood estimation approach.

TND COVID-19 Vaccine Effectiveness Using Ordinary Logistic Regression

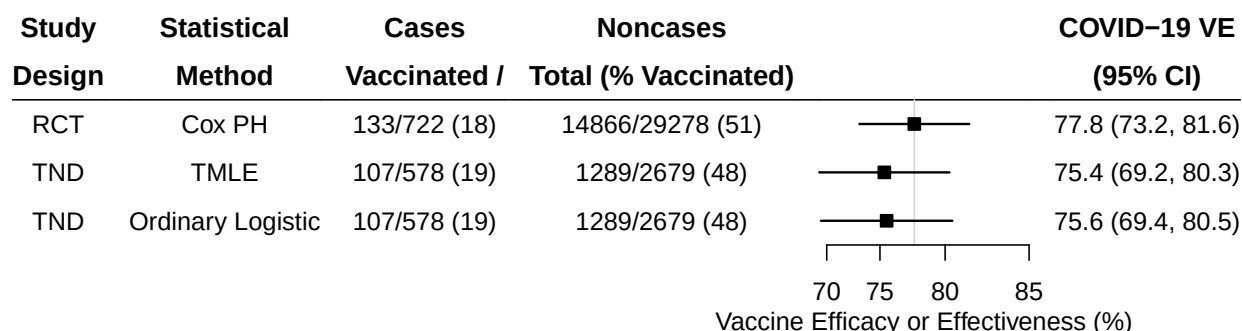
We also estimate the TND sample odds ratio of COVID-19 adjusted for vaccination status, age, sex assigned at birth, race/ethnicity, region, comorbidities, and two-week calendar time interval main effects using an ordinary logistic regression.

```
ord.mod <- glm(POSTEST ~ VACCINE + AGE + SEX + POC + REGION +
               COMORB + TWOWEEK, data = part.wcc.df,
               family = "binomial")
```

TND vaccine effectiveness against symptomatic COVID-19 in healthcare-seeking individuals with the same covariates using the ordinary logistic regression is 75.6% (95% CI: 69.4, 80.5).

Summary of Results

We use forest plots to visualize the COVID-19 RCT vaccine efficacy and TND vaccine effectiveness estimates together.



TND vaccine effectiveness estimates from both statistical methods are highly concordant with the RCT vaccine efficacy estimates. The TND estimates' confidence intervals are slightly wider than the RCT estimates' confidence intervals, largely due to the difference in sample size. All confidence intervals overlap.

References

1. El Sahly HM, Baden LR, Essink B, et al. Efficacy of the mRNA-1273 SARS-CoV-2 Vaccine at Completion of Blinded Phase. *N Engl J Med*. 2021;385(19):1774-1785. doi:10.1056/NEJMoa2113017
2. De Serres G, Skowronski DM, Wu XW, Ambrose CS. The test-negative design: validity, accuracy and precision of vaccine efficacy estimates compared to the gold standard of randomised placebo-controlled clinical trials. *Eurosurveillance*. 2013;18(37). doi:10.2807/1560-7917.ES2013.18.37.20585
3. Andrews LIB, Halloran ME, Neuzil KM, et al. Evaluating the Test-Negative Design for COVID-19 Vaccine Effectiveness Using Randomized Trial Data: A Secondary Cross-Protocol Analysis of 5 Randomized Clinical Trials. *JAMA Network Open*. 2025;8(5):e2512763. doi:10.1001/jamanetworkopen.2025.12763
4. van der Laan L. *causalglm*: Interpretable and robust causal inference for heterogeneous treatment effects using generalized linear models with targeted machine-learning. Published online June 2, 2024. Accessed August 27, 2024. <https://github.com/tlverse/causalglm>